Team: Bid Data Big Team

Business Goals

The primary objective of this project is to transform operational data from the Northwind source system into a structured dimensional model (Star Schema). This enables the business to:

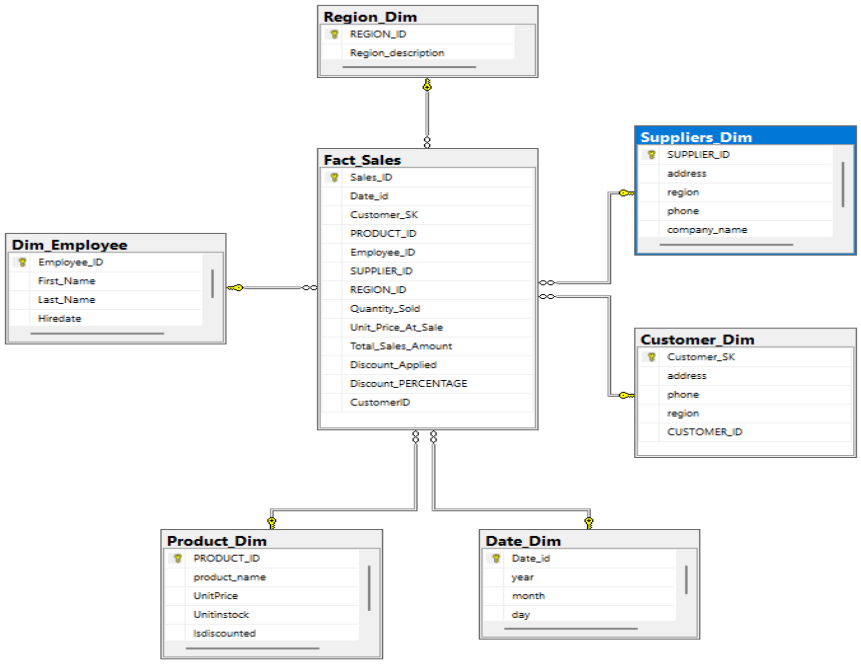
- Analyze sales performance across different time dimensions (Year, Month, Day).
- Calculate and know the total sales in a certain month
- Monitor product inventory levels and supplier contributions to revenue.
- Identify customer purchasing patterns and discount impact on total sales.

Source ERD/Schema (Interested Tables)

The data is extracted from the Northwind source and moved into a staging environment (staging_project1) before being loaded into the final Data Warehouse. The core entities involved in this process include:

- **Stg_Sales:** The primary source for transactional facts.
- **Stg_Customer, Stg_Product, Stg_Employee, Stg_Supplier, Stg_Region:** .

d. Star Schema (Dimensional Model)



The diagram illustrates the Star Schema designed for the ABschema database. It features a centralized Fact table (Fact_Sales) connected to six Dimension tables.

Schema Description

I. Dimensions

- **Date_Dim:** Captures the temporal aspects of a sale (Year, Month, Day).
- **Customer_Dim:** Stores customer details,for historical tracking and the natural CUSTOMER_ID.
- **Product_Dim:** Contains product specifications, unit prices, and stock status.
- **Dim_Employee:** Maintains records of the sales staff involved in transactions.
- **Suppliers_Dim:** Provides context regarding the origin of the products sold.
- **Region_Dim:** Include Region id and region description.

II. Fact Table

- **Fact_Sales:** This is the central table of the schema. It stores the foreign keys connecting to all dimensions and the quantitative measures of the business process.

III. Measures

The following quantitative metrics are available in the Fact_Sales table for analysis:

- **Quantity_Sold:** The total number of units sold .
- **Unit_Price_At_Sale:**The price of the product after the discount.
- **Total_Sales_Amount:** The calculated revenue .
- **Discount_PERCENTAGE:** The percentage of discount applied to the sale.

i.How we handled nulls

We used derived column to handle the null values for example in quantity sold if it has null values the derived column change it to zero same as discount percentage it will be 0 if it is

empty.

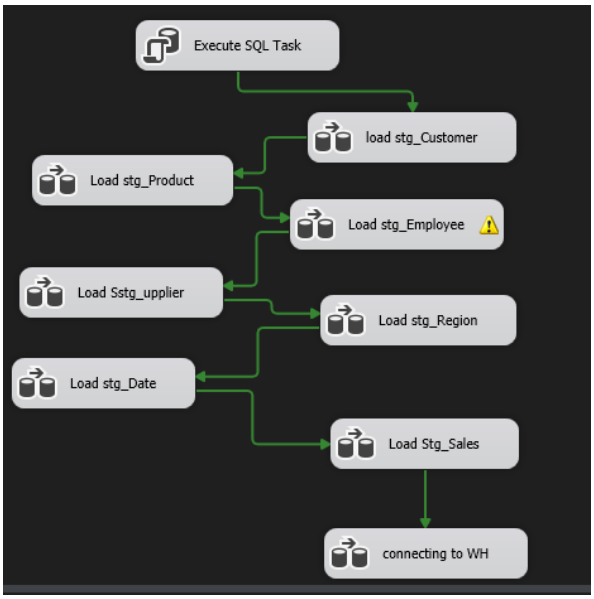
Derived Column Name	Derived Column	Expression	Data Type
Total_Sales_Amount	<add as new column>	der_Quantity_Sold * Unit_Price_At_Sale * (1 - der_Di...	float [DT_R4]
der_Quantity_Sold	Replace 'der_Quantity_...	REPLACENULL(der_Quantity_Sold,0)	two-byte signed intege...
der_Discount_PERCEN...	Replace 'der_Discount_...	REPLACENULL(der_Discount_PERCENTAGE,0)	float [DT_R4]

ii. Handing duplicates

For duplicates we used look up no match with the destination to see if the the primary key like product id or employee id if it is already in the destination column it will not be added to the table.

iii. Handing more than one run error

we added a sql command that remove the data in the staging table and at the end we connect the stage fact table as a source to the DW fact table as destination with lookup (No match)



iv.Mdx queries we used

```sql
-- for query one-->
SELECT ProductName, UnitPrice
FROM Stg_Product;
-- query 2 -->
SELECT
    p.ProductName,
    SUM(s.Total_Sales_Amount) AS TotalRevenue,
    s.Unit_Price_At_Sale * (1 - s.Discount_PERCENTAGE) AS PriceAfterDiscount
FROM Stg_Sales s
JOIN Stg_Product p ON s.ProductID = p.ProductID
WHERE s.Discount_PERCENTAGE > 0
GROUP BY p.ProductName, s.Unit_Price_At_Sale, s.Discount_PERCENTAGE;
```

Query 1 Product Price List

What it does: This query retrieves a basic list of all products

What it gets: It shows two specific columns: the **Product Name** and its standard **Unit Price**.

Purpose: To provide a simple overview of the current catalog prices.

Query 2 Discounted Sales Performance

What it does: This query looks at sales where a discount was applied and calculates the financial outcome.

What it gets:

- **ProductName:** The name of the item sold.
- **TotalRevenue:** The sum of all sales amounts for that specific item.
- **PriceAfterDiscount:** The actual price the customer paid after the discount was taken off.

Purpose: To analyze how much revenue is being generated from items that are on sale.

```sql
--query 3-->
SELECT
    c.Region AS RegionName,
    p.ProductName,
    SUM(s.Quantity_Sold) AS TotalQty
FROM Stg_Sales s
JOIN Stg_Product p ON s.ProductID = p.ProductID
JOIN Stg_Customer c ON s.CustomerID = c.CustomerID
GROUP BY c.Region, p.ProductName;
--Query 4 ->
SELECT
    c.Region AS RegionName,
    SUM(s.Quantity_Sold) AS TotalItemsSold
FROM Stg_Sales s
JOIN Stg_Customer c ON s.CustomerID = c.CustomerID
WHERE c.Region IS NOT NULL
GROUP BY c.Region;
```

Query 3 Product Sales by Region

What it does: This query combines sales, product, and customer data to see which items are popular in different areas.

What it gets: It lists the **Region Name**, the **Product Name**, and the **Total Quantity** of that product sold in that specific region.

Purpose: To identify regional demand for specific products.

Query 4 Total Volume per Region

What it does: Get the total items sold for each region.

What it gets: It shows each **Region Name** and the **Total Items Sold** (sum of all quantities) across all products for that region.

Purpose: To compare the overall sales performance and volume between different geographical regions.

```sql
--query 5 -->
SELECT
    t1.CustomerID,
    t1.Address AS HistoricalAddress, -- The address at the time of the transaction
    t2.CurrentAddress,               -- The address they have right now
    t1.Phone AS HistoricalPhone,     -- The phone at the time of the transaction
    t2.CurrentPhone,                 -- The phone they have right now
    t1.Region
FROM Stg_Customer t1
JOIN (
    -- This subquery finds the "latest" record for each customer
    SELECT
        CustomerID,
        Address AS CurrentAddress,
        Phone AS CurrentPhone
    FROM (
        SELECT CustomerID, Address, Phone,
               ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY (SELECT NULL)) as Latest
        FROM Stg_Customer
    ) tmp
    WHERE Latest = 1
) t2 ON t1.CustomerID = t2.CustomerID;
```

Query 5 Customer Contact History (SCD Type 2 Tracking)

What it does: This query compares a customer's historical information with their most recent records.

What it gets: It displays the **Customer ID** alongside their **Historical Address/Phone** (from the time of a past transaction) and their **Current Address/Phone** (the most up-to-date info).

Purpose: To track changes in customer contact details over time, which is essential for maintaining accurate historical data in the warehouse.

```sql
--query 6 -->
WITH MonthlyStats AS (
    -- Step 1: Calculate total quantity and revenue for every month
    SELECT
        MONTH(DateID) as SaleMonth,
        SUM(Quantity_Sold) as MonthlyQty,
        SUM(Total_Sales_Amount) as MonthlyRevenue
    FROM Stg_Sales
    GROUP BY MONTH(DateID)
),
TopMonth AS (
    -- Step 2: Find the single month with the highest Quantity_Sold
    SELECT TOP 1 SaleMonth, MonthlyRevenue
    FROM MonthlyStats
    ORDER BY MonthlyQty DESC
)
-- Step 3: Pull detailed transaction data for that specific top month
SELECT
    s.SalesTransactionID AS SalesID,
    s.DateID,
    MONTH(s.DateID) AS MonthNumber,
    p.ProductName,
    s.CustomerID AS Customer_ID, -- Note: Stg_Customer doesn't have a 'Name' column
    s.UnitPrice AS StandardUnitPrice, -- From the Sales table
    s.Discount_PERCENTAGE,
    -- Calculation: Price after applying the discount
    (s.Unit_Price_At_Sale * (1 - ISNULL(s.Discount_PERCENTAGE, 0))) AS PriceAfterDiscount,
    s.Total_Sales_Amount AS TransactionTotalSales,
    tm.MonthlyRevenue AS TotalRevenueForThisMonth
FROM Stg_Sales s
JOIN Stg_Product p ON s.ProductID = p.ProductID
JOIN TopMonth tm ON MONTH(s.DateID) = tm.SaleMonth
WHERE MONTH(s.DateID) = tm.SaleMonth;
```

Query 6 Top Sales Month Deep-Dive

What it does: This is a query first identifies the single month with the highest quantity of items sold and then pulls every individual transaction that happened during that specific month.

What it gets: Detailed transaction info like **SalesID**, **ProductName**, and **Customer_ID**.

- Financial calculations like the **PriceAfterDiscount** for each item.
- The total revenue for that entire "top month" compared to individual sales.

Purpose: To perform an analysis into the business's most successful month to understand what drove that peak performance.

```
--Query 7 -->
SELECT
    MONTH(DateID) AS MonthNumber,
    SUM(Quantity_Sold) AS TotalItemsSold,
    SUM(Total_Sales_Amount) AS TotalRevenue
FROM Stg_Sales
GROUP BY MONTH(DateID)
ORDER BY MonthNumber;
```

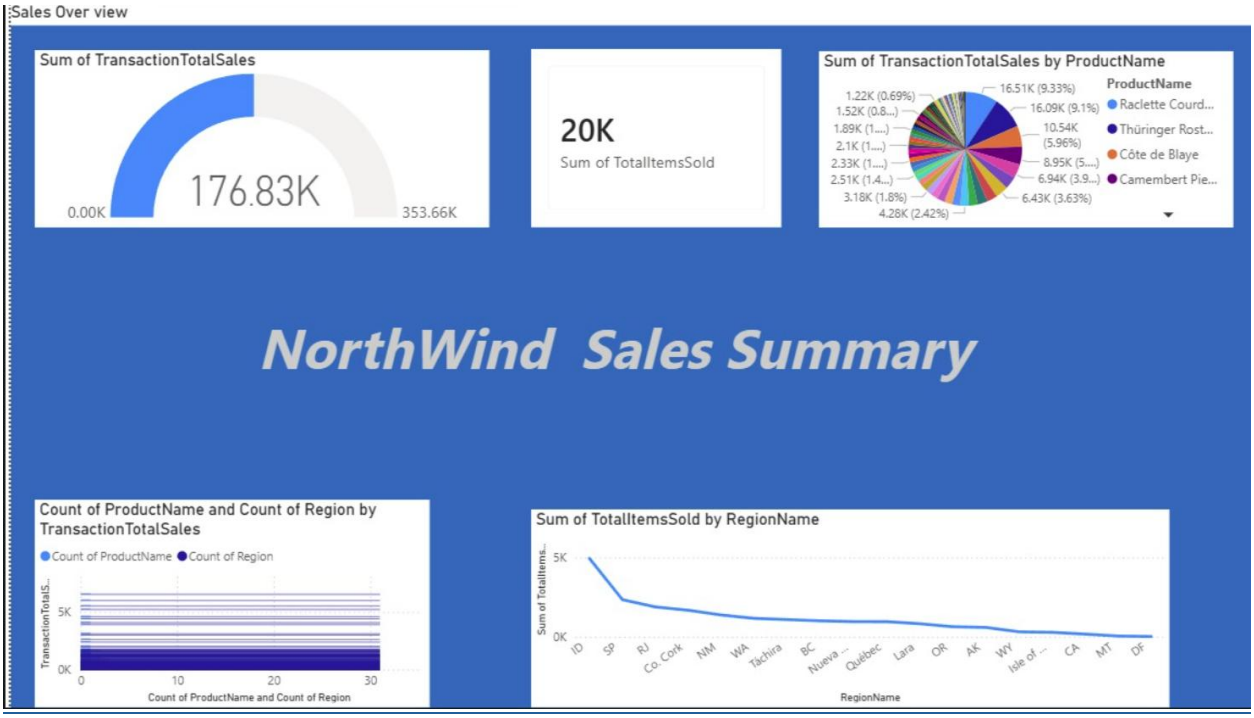
Query 7 Monthly Sales Trends

What it does: This query summarizes business performance by grouping all sales data into calendar months.

What it gets: It shows the **Month Number**, the **Total Items Sold**, and the **Total Revenue** generated for each month of the year.

Purpose: To track growth trends throughout the year and identify seasonal patterns in sales volume and income.

Power Pi


Sales Over view
Sum of TransactionTotalSales
176.83K
0.00K
353.66K
20K
Sum of TotalItemsSold
Sum of TransactionTotalSales by ProductName
ProductName
Raclette Courd...
Thüringer Rost...
Côte de Blaye
Camembert Pie...
16.51K (9.33%)
16.09K (9.1%)
10.54K (5.96%)
8.95K (5....)
6.94K (3.9...)
6.43K (3.63%)
4.28K (2.42%)
3.18K (1.8%)
2.51K (1.4...)
2.33K (1....)
2.1K (1....)
1.89K (1....)
1.52K (0.8...)
1.22K (0.69%)
NorthWind Sales Summary
Count of ProductName and Count of Region by TransactionTotalSales
Count of ProductName
Count of Region
TransactionTotalS...
5K
0K
0
10
20
30
Count of ProductName and Count of Region
Sum of TotalItemsSold by RegionName
Sum of TotalItems...
5K
0K
ID
SP
RJ
Co. Cork
NM
WA
Táchira
BC
Nueva ...
Québec
Lara
OR
AK
WY
Isle of ...
CA
MT
DF
RegionName